

Universidad Nacional de Rosario

Facultad de Ciencias Exactas, Ingeniería y Agrimensura

Trabajo Práctico Integrador

Sistemas embebidos avanzados II

Alumno: RAMOS, Nicolás

Legajo: R-3892/1

Año: 2026

Índice

1. Introducción

2. Validación de hardware placa de desarrollo

2.1 Objetivo y contexto

2.2 Descripción del hardware

2.3 Sketch de prueba

2.4 Herramientas de desarrollo

3. Etapa de modelado (UML)

3.1 Diagrama de casos de uso

3.2 Diagrama de actividad

3.3 Diagrama de despliegue

3.4 Diagrama de máquina de estados

3.5 Diagrama de componentes

3.6 Diagrama de secuencia

4. Etapa de implementación

4.1 Plataforma y herramientas

4.2 Estructura del proyecto y organización del código

4.3 Tareas y prioridades

4.4 Mecanismos de sincronización

4.5 Manejo de errores

4.6 Consideraciones de concurrencia

5. Conclusiones

6. Referencias y código fuente

1. Introducción

Este informe documenta de forma integrada el trabajo práctico desarrollado en la asignatura Sistemas Embebidos Avanzados II. El trabajo se organizó en dos etapas complementarias.

En la primera etapa se realizó el montaje de componentes sobre la placa de desarrollo para aplicaciones industriales IOT basada en ESP32 y la validación de hardware. El objetivo fue comprobar el correcto funcionamiento de entradas y salidas digitales, entradas analógicas, salidas DAC, display OLED, buzzer y periféricos asociados, antes de abordar el firmware más exigente en tiempo real.

En la segunda etapa se implementó un control de temperatura aplicado a una caldera, ejecutado sobre el sistema operativo en tiempo real FreeRTOS incluido en ESP-IDF. La información del proceso se presenta en un display OLED y en una página web servida por el propio microcontrolador, configurando el WiFi del ESP32 como punto de acceso (AP) para acceder sin depender de una red externa.

2. Validación de Hardware

2.1 Objetivo y contexto

El proyecto consistió en llevar a cabo el montaje de componentes, relevamiento del circuito esquemático y la validación de todas las etapas del circuito de forma individual. Una vez realizada dicha validación, se diseñó una plantilla para inicializar y configurar periféricos de la placa de desarrollo y una librería para el uso del display OLED, para ser usadas por futuros alumnos de la asignatura.

La placa incorpora periféricos útiles para automatización: entradas digitales y analógicas, salidas a relé y triacs, salidas PWM/DAC, display OLED por I²C, módulo RTC, buzzer y salidas de potencia. Para esta etapa se utilizó el IDE Arduino con el core ESP32, por la rapidez para compilar, flashear y probar cada recurso sin la sobrecarga inicial de un proyecto ESP-IDF completo.

2.2 Descripción del hardware

A continuación se resume la correspondencia entre recursos de la placa y GPIO del ESP32, coherente con el sketch de validación PLC_ESP32_EntradasSalidas.ino.

2.2.1 Entradas digitales (I0.0 – I0.3)

Cuatro entradas digitales aisladas mediante optoacopladores, conectadas a GPIO del ESP32. En las pruebas se habilitaron pull-down internas.

Entrada	GPIO ESP32
I0.0	5
I0.1	15
I0.2	16
I0.3	34

Tabla 1 — Entradas digitales y GPIO.

2.2.2 Salidas digitales (Q0.0 – Q0.7)

Tres salidas a relés (Q0.0–Q0.2) y cuatro salidas a triacs (Q0.3–Q0.7), con etapas de acoplamiento.

Salida	GPIO ESP32
Q0.0	4
Q0.1	17
Q0.2	18
Q0.3	19

Salida	GPIO ESP32
Q0.4	23
Q0.5	27
Q0.6	32
Q0.7	33

Tabla 2 — Salidas digitales y GPIO.

2.2.3 Entradas analógicas (A0.0 – A0.1)

Entrada	GPIO
A0.0	36
A0.1	35

Tabla 3 — Entradas analógicas.

2.2.4 Salidas DAC (DAC0.0 – DAC0.1)

Salida	GPIO
DAC0.0	25
DAC0.1	26

Tabla 4 — Salidas DAC del ESP32.

2.2.5 Display OLED (I²C)

Display compatible con controlador SH1106, típicamente 128×64, dirección I²C 0x3C, SDA GPIO 21 y SCL

GPIO 22.

2.2.6 Buzzer y LED RGB

Buzzer en GPIO 2. LED RGB (cátodo común): R=14, G=13, B=12, según definiciones del sketch de validación.

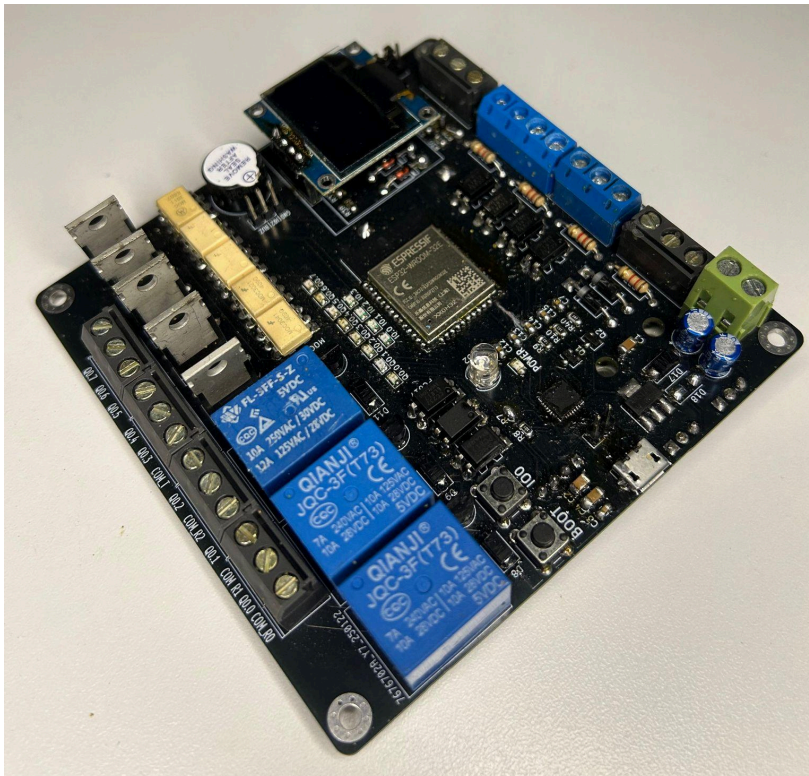


Figura 1. Placa de desarrollo para aplicaciones industriales IoT basada en ESP32

2.3 Sketch prueba de hardware

Con el programa de ejemplo se verificó, entre otros puntos:

- **Entradas digitales I0.0–I0.3:** lectura coherente en reposo y al activar cada entrada, con pull-down internas habilitadas.
- **Entradas Analógicas A0.0–A0.1:** lectura con analogRead y envío por puerto serie para correlacionar tensión aplicada (0–3,3 V) con el valor digital.
- **Salidas digitales Q0.0–Q0.7:** conmutación secuencial e individual; coherencia entre estado lógico y actuación sobre LED, relé o carga de prueba.

- **Salidas DAC:** uso de `dacWrite` y comprobación con multímetro.
- **OLED:** inicialización con librería Adafruit SH110X; texto y actualización periódica.
- **Buzzer:** conmutación periódica del pin para verificar audio; puede extenderse con `tone()`.

2.4 Herramientas de desarrollo

- IDE Arduino con soporte ESP32, para el sketch `PLC_ESP32_EntradasSalidas.ino`.
- Cable USB y monitor serie para depuración y registro de valores analógicos.

3. Etapa de modelado (UML)

En esta parte del trabajo práctico se trabajó en un control de temperatura aplicado a una caldera, basado en sistema operativo en tiempo real, con varias actividades concurrentes, recursos compartidos y restricciones temporales.

Antes de escribir o integrar el firmware definitivo, se necesitó disponer de un modelo formal del sistema. Para ello, se utilizó como lenguaje común entre requisitos, diseño e implementación, el lenguaje UML (*Unified Modeling Language*). El modelado obliga a nombrar actores, límites del sistema, secuencias de interacción, estados discretos del control y piezas de software que deben coexistir bajo el planificador. A continuación se mostrarán los diagramas correspondientes para describir la lógica y dinámica del sistema, arquitectura del software y disposición del hardware.

3.1 Diagrama de casos de uso

El diagrama de casos de uso presenta el sistema embebido como una caja negra y enumera las capacidades visibles desde el exterior, sin entrar en tareas ni mutex. El límite del sistema (*system boundary*) encierra al controlador sobre la placa de desarrollo.

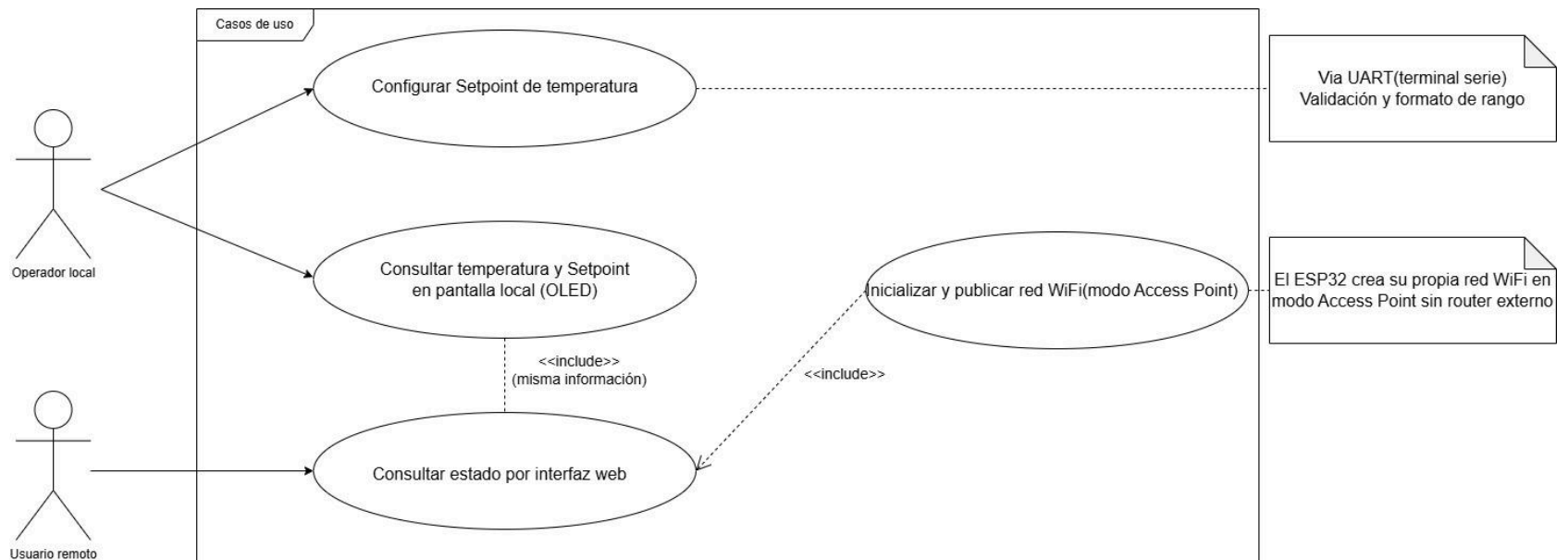


Figura 2. Diagrama Casos de Uso

3.2 Diagrama de actividad

El diagrama de actividad describe **flujos de control** con decisiones, bifurcaciones y actividades que pueden ser paralelas o secuenciales.

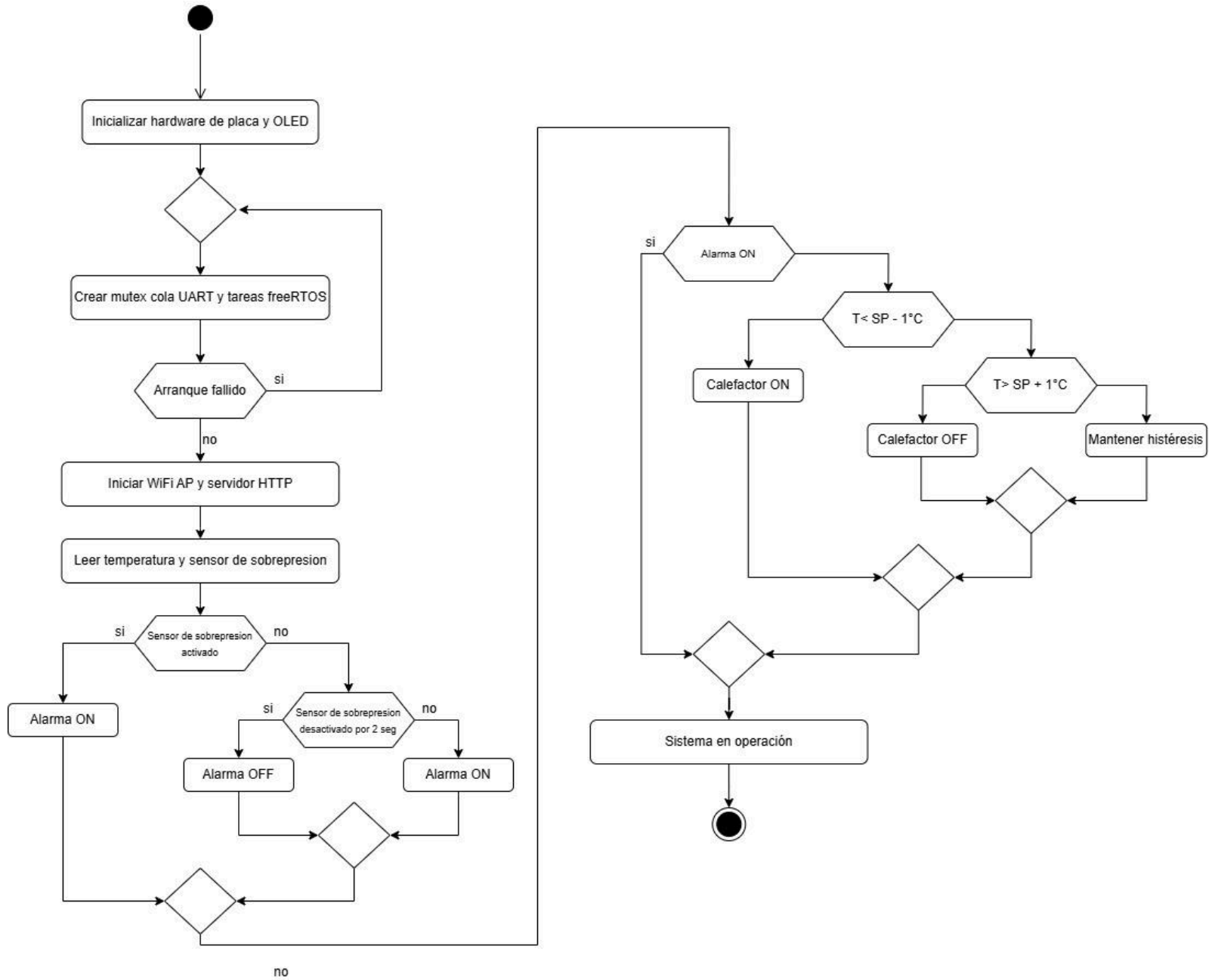


Figura 3. Diagrama de actividad

3.3 Diagrama de despliegue

Este diagrama se utiliza para modelar la arquitectura física de un sistema. Su objetivo principal es mostrar cómo se distribuyen los componentes de software en los recursos de hardware (como servidores, computadoras o dispositivos móviles) durante el tiempo de ejecución

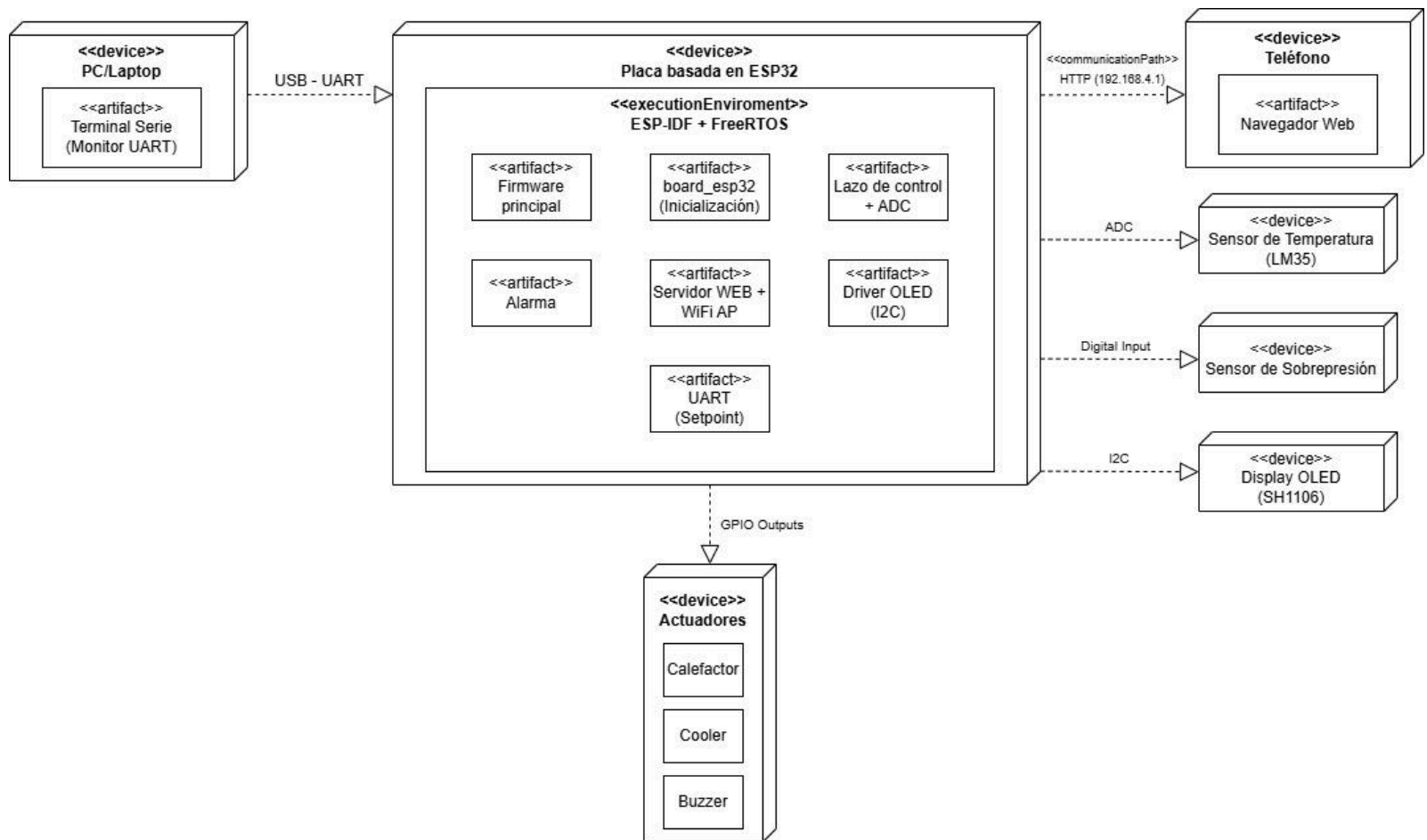


Figura 4. Diagrama de despliegue

3.4 Diagrama Máquinas de Estado

Este diagrama describe cómo se comporta un objeto específico o un sistema entero basándose en su historial de eventos. El sistema posee al menos dos estados lógicos dominantes: Normal y Alarma por sobrepresión. En Normal, la calefacción sigue una lógica con histéresis alrededor del setpoint y el cooler permanece desactivado salvo políticas adicionales no usadas aquí. En Alarma, la calefacción queda forzada a apagarse, el cooler se activa, el buzzer se activa de forma intermitente y las interfaces muestran un mensaje de advertencia.

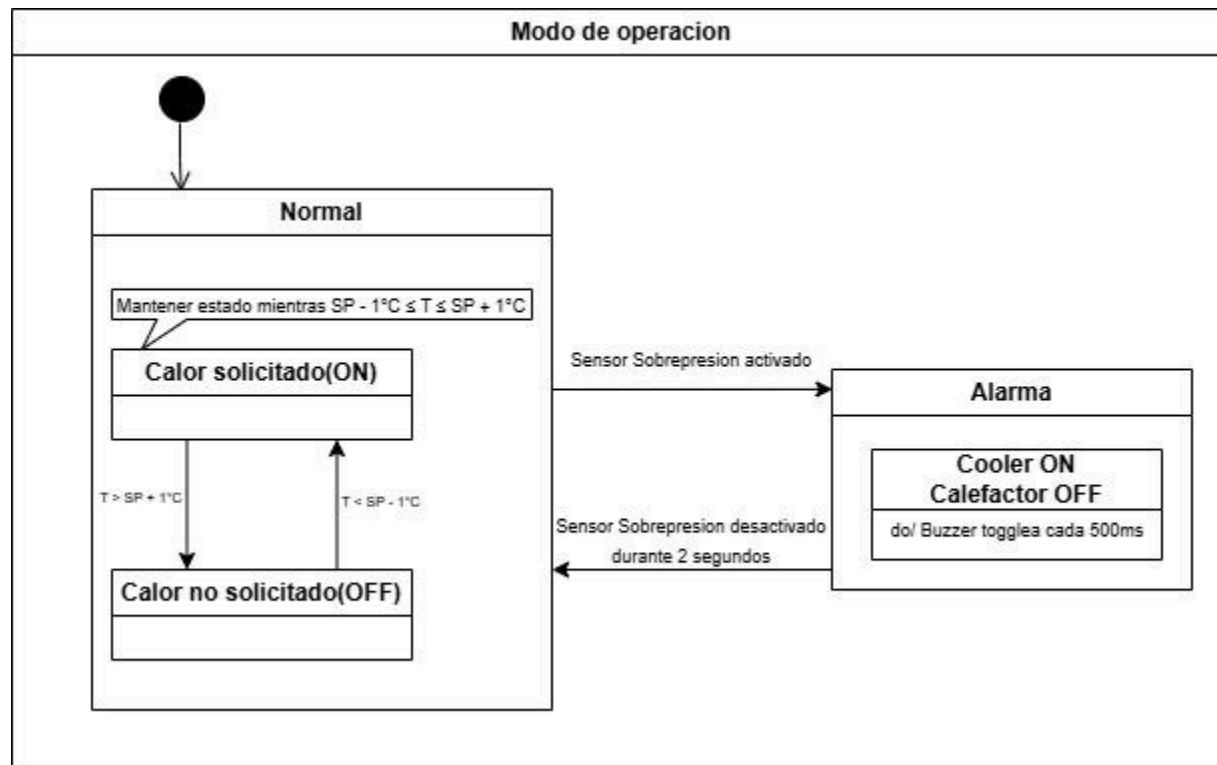


Figura 5. Diagrama de Máquinas de Estado

3.5 Diagrama de componentes

El diagrama de componentes muestra la arquitectura de software y las dependencias entre piezas, alineado con lo que luego se implementará como archivos y tareas.

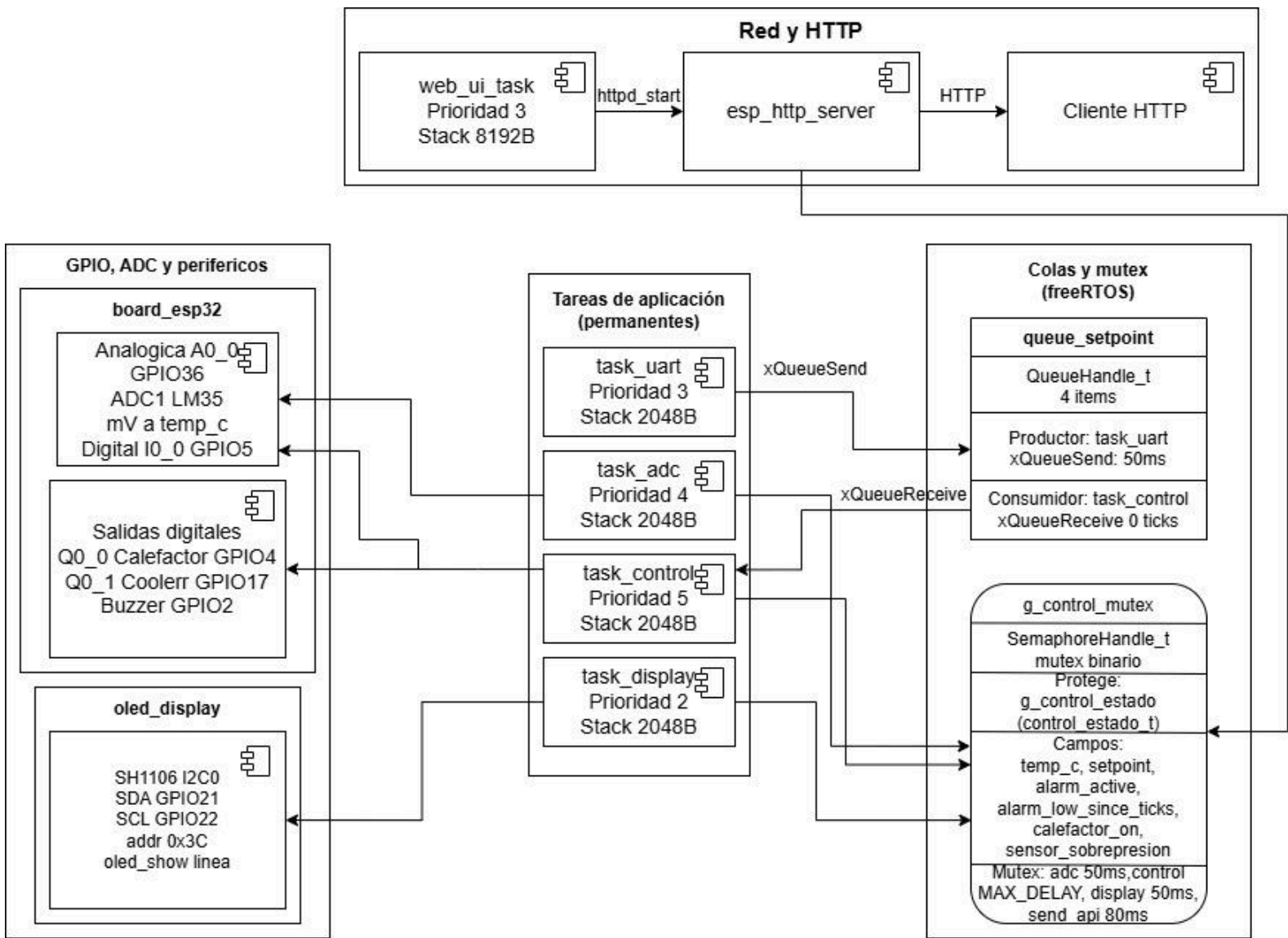


Figura 6. Diagrama de componentes

Entre estos componentes aparecen interfaces de FreeRTOS: una cola de mensajes para transportar enteros de setpoint desde la UART hacia el control; un mutex asociado a la estructura de estado compartido que deben

respetar la adquisición, el control, la vista OLED y el endpoint JSON. El diagrama debe mostrar que el servidor HTTP *depende* del mismo tipo de datos que la tarea de display, pero no debe dibujarse como si el HTTP escribiera directamente temperatura: en la implementación, el HTTP solo lee una copia bajo mutex. Las flechas de dependencia van desde las tareas o componentes de aplicación hacia los drivers de ESP-IDF (UART, GPIO, ADC, esp_wifi, esp_http_server), indicando que el firmware de aplicación se apoya en el framework.

3.6 Diagrama de secuencia

Es un diagrama de comportamiento que modela la interacción lógica entre objetos, enfocándose específicamente en el orden temporal en el que se intercambian mensajes para ejecutar una funcionalidad o escenario concreto. A diferencia de los diagramas estáticos (como el de clases de uso), este es dinámico: describe cómo "vive" y se comunica el sistema paso a paso durante un proceso.

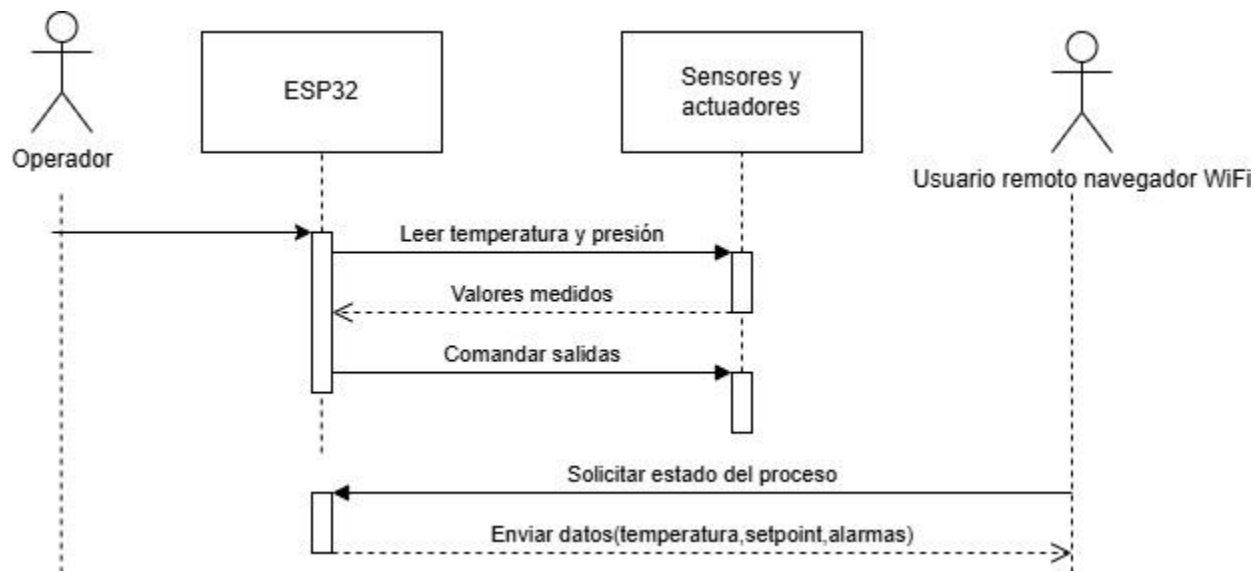


Figura 7. Diagrama de secuencia

4. Etapa de implementación

4.1 Plataforma y herramientas

La implementación del proyecto se basa en una placa de desarrollo equipada con el microcontrolador ESP32. Para su programación se utilizó el framework oficial del fabricante ESP-IDF, el cual integra el kernel del sistema operativo en tiempo real FreeRTOS y pone a disposición toda su API (tareas, colas, semáforos, mutex). El flujo de trabajo se gestionó a través del entorno Visual Studio Code con la extensión ESP-IDF, permitiendo compilar, flashear y monitorear el dispositivo por el puerto serial (UART0, vía USB). La UART0 queda reservada para la configuración del setpoint.

Para esta parte del trabajo práctico, se recurrió a la herramienta Cursor el cual es un entorno de programación basado en Visual Studio Code que integra asistencia con modelos de lenguaje (inteligencia artificial) para tareas como completar código, explicar fragmentos y apoyar la revisión de errores de compilación o de sintaxis. En ningún caso se delegó el criterio de diseño ni la responsabilidad del trabajo al asistente, el uso fue incremental y supervisado, es decir, se le indicaron objetivos concretos, se corrigió o descartó lo que no encajaba con el enunciado o con el hardware real, y se fue guiando el resultado (prioridades de tareas, sincronización con mutex y colas, comportamiento de alarmas, etc.) hasta alinearlos con lo implementado en el firmware y con lo exigido por la asignatura. Básicamente su uso sirvió para optimizar tiempos de codificación, por ejemplo en la estructuración de módulos y partes repetitivas del código, pero el contenido técnico, las decisiones y la verificación quedaron a mi cargo.

4.2 Estructura del proyecto y organización del código

El proyecto se organiza en módulos con responsabilidades bien definidas. A continuación presento los archivos y su función dentro del proyecto:

ESP32_FreeRTOS: Archivo principal del firmware. Contiene el punto de entrada `app_main()`, la definición de la estructura de estado compartido (`estado_t`), el mutex y la cola, las constantes de prioridad y tiempos, las

cuatro tareas: `task_adc`, `task_uart`, `task_control`, `task_display`, y `web_ui_start()`. Incluye además la función auxiliar `leer_temperatura_lm35()` que convierte la lectura del ADC a temperatura en °C.

board_esp32.c: Implementación de la capa de entradas/salidas. Configuración de GPIOs y ADC1.

board_esp32.h: Cabecera del módulo `board_esp32`. Define los números de GPIO para todas las entradas y salidas.

oled_display.c: Driver del display OLED SH1106 por I2C. Implementa inicialización del bus I2C (SDA y SCL), secuencia de inicio del SH1106, y las funciones `oled`. Utiliza una fuente 6×8 para mostrar texto centrado en una línea.

oled_display.h: Cabecera del módulo OLED. Define pines I2C, dirección del dispositivo (0x3C), dimensiones del display y declara las funciones públicas.

control_state.c: Implementación mínima del módulo. Inicializa `g_control_estado` con valores por defecto (setpoint, flags), aquí no se crea el mutex, sólo concentra el dato compartido del sistema.

control_state.h: Cabecera del estado compartido del control. Define el struct `control_estado_t` (temperatura, setpoint, alarma, flags y sobrepresión), y declara variables globales `g_control_estado` y el handle `g_control_mutex` para que las tareas y el HTTP las usen de forma coherente.

web_ui.c: Implementa el modo WiFi AP (NVS, netif, credenciales), arranca `httpd` con las rutas `/html` y `/api/status` (JSON leyendo el estado bajo mutex) y crea la tarea `web_ui_task` que hace el init y luego se elimina mientras el servidor sigue activo.

web_ui.h: Cabecera del servidor web embebido. Declara la función de arranque `web_ui_start` para levantar el punto de acceso WiFi y el servidor HTTP.

4.3 Tareas y prioridades

Las tareas de aplicación son hasta cinco: cuatro se crean con `xTaskCreate` en `app_main()` (la de UART sólo si el driver instaló bien) y la quinta, `web_ui_task`, se crea dentro de `web_ui_start()` llamada desde `app_main()`. Las cuatro tareas periódicas corren en bucle infinito y ceden el CPU con `vTaskDelay`, mutex o cola/UART bloqueante; `web_ui_task` ejecuta la secuencia de arranque de red y HTTP y termina con `vTaskDelete`.

Tarea de adquisición de temperatura (task_adc)

Su función es mantener actualizado el valor de temperatura del proceso en la estructura de estado compartido, a partir de la lectura periódica del canal analógico A0.0 conectado al sensor LM35. No actúa sobre periféricos de salida, sólo escribe el campo temp_c del estado_t. En cada iteración la tarea intenta tomar el mutex con un timeout de 50 ms. Si lo obtiene, escribe la temperatura recién leída en estado.temp_c y libera el mutex de inmediato; en caso contrario omite la actualización en ese ciclo. A continuación ejecuta vTaskDelay(200 ms), con lo que cede el procesador y fija el período efectivo de muestreo en 200 ms.

La tarea se ejecuta en un bucle infinito. No mantiene estado local entre iteraciones más allá de la variable de temperatura leída. Ante fallo del ADC, leer_temperatura_lm35() retorna 0.0 °C, por lo que la tarea sigue actualizando el estado sin bloquear el sistema.

Tarea de recepción de setpoint por UART (task_uart)

Su función es encargarse de recibir por UART0 los setpoints que envía el operador desde un terminal o adaptador serial. Funciona en un bucle continuo y, en cada iteración, lee un solo byte de forma bloqueante (portMAX_DELAY), porque el enlace USB suele entregar los caracteres de a uno y leer bloques fijos de dos bytes no es fiable. Esos caracteres se van acumulando en un buffer interno hasta detectar un fin de línea (\r o \n). En ese momento se interpreta la línea: si contiene exactamente un dígito ASCII entre '0' y '9', el setpoint resultante es ese valor (0–9); si tiene al menos dos caracteres y los dos primeros son dígitos, se forma el número en decimal como decenas $\times$ 10 más unidades y se acota al rango 0–99 (valores mayores a 99 se consideran 99). Cualquier otra forma de línea al pulsar Enter no produce un envío a la cola. Cuando hay un setpoint válido, la tarea lo pone a disposición de task_control mediante xQueueSend sobre la cola de FreeRTOS, sin usar mutex, ya que la API de colas serializa de forma segura el paso de mensajes entre tareas. Si la cola está llena, la llamada espera como máximo 50 ms; si en ese intervalo no se pudo encolar, el intento falla, el valor de esa línea no se conserva para un reenvío posterior y la tarea sigue ejecutándose sin quedar bloqueada indefinidamente. La tarea no modifica el setpoint por defecto del sistema ni escribe directamente el estado compartido: solo aporta nuevos valores cuando el operador envía explícitamente una línea válida por UART0, y es task_control quien incorpora esos valores a partir de lo recibido por la cola.

Tarea de control (task_control)

Esta tarea concentra la lógica del lazo de temperatura y de la máquina de estados asociada a la sobrepresión leída en I0.0. En cada ciclo, primero intenta incorporar un nuevo setpoint desde la cola FreeRTOS con xQueueReceive y tiempo de espera nulo, de modo que no bloquee si no hay mensajes pendientes. A continuación toma el mutex g_control_mutex con portMAX_DELAY, es decir, espera el tiempo que haga falta hasta entrar en la sección crítica, y no hay un segundo bloqueo ni timeouts cortos dedicados al setpoint. Dentro de esa misma sección se lee la temperatura ya publicada en el estado global, se interpreta I0.0 según la polaridad configurada (PRESION_I0_ACTIVADO_NIVEL_ALTO), se actualiza el indicador de alarma activa (entrada inmediata cuando la presión indica falla y salida solo tras 2000 ms continuos con I0.0 en el nivel “inactivo”), se escribe setpoint en el estado compartido para que la tarea de display y la interfaz web muestren el valor vigente, y se calcula el pedido de calefacción con histéresis respecto del setpoint, salvo en alarma, donde se anula la retención del calefactor. Tras xSemaphoreGive, fuera de la sección crítica, se aplican las salidas físicas: en alarma se apaga Q0.0(calefactor), se enciende Q0.1(cooler) y se hace togglear el buzzer con un período fijo; en operación normal se apagan cooler y buzzer y se activa Q0.0 según el resultado del control con histéresis. El ritmo del lazo queda acotado por vTaskDelay(100 ms) al final de cada iteración.

Tarea de presentación en display (task_display)

Esta tarea cumple una función exclusivamente de interfaz humano - máquina: pone a disposición del operador, en el display OLED, la información esencial del proceso, la temperatura medida, el setpoint vigente y la condición de alarma de sobrepresión. Cuando la alarma está activa, el mensaje se reduce a una leyenda fija de Sobrepresión; en operación normal se arma una cadena con temperatura y setpoint y se envía al driver del OLED. Para obtener esos datos sin mezclar lecturas parciales con escrituras concurrentes de otras tareas, la rutina toma el mutex con un timeout de 50 ms, copia en variables locales la temperatura, el setpoint y el flag de alarma, y libera de inmediato el mutex. Ejecuta el trabajo más lento —formateo de texto y actualización del display vía I2C— fuera de la sección crítica, lo que mantiene acotado el tiempo en que el mutex está tomado y reduce la interferencia con tareas que actualizan el mismo estado global. Si en un ciclo no se consigue el mutex dentro del plazo de espera, la tarea no refresca el OLED en esa pasada, aplica un vTaskDelay de 200

ms y vuelve a intentar en la siguiente vuelta del bucle, evitando bloqueos indefinidos. Tras una actualización exitosa del display, el ritmo queda marcado por un vTaskDelay de 500 ms, de modo que la presentación se renueva aproximadamente dos veces por segundo. La tarea no modifica el estado del control ni las salidas de campo; se limita a leer el estado compartido y a reflejarlo en pantalla.

Tarea de interfaz web (web_ui_task)

Es la encargada de poner en marcha la interfaz web del proyecto. Se crea desde web_ui_start() con prioridad 3 y un stack de 8192 Bytes, pensada para absorber el consumo de memoria del init de WiFi en modo punto de acceso (NVS, esp_netif, bucle de eventos, configuración del AP con SSID y contraseña fijos) y la creación del servidor HTTP del ESP-IDF. En la práctica, la rutina inicializa el AP, espera 500 ms para dejar estabilizar la interfaz de red y llama a httpd_start, registrando dos URL: la raíz que sirve la página HTML embebida con estilos y un script que consulta periódicamente el estado, y /api/status, que responde en JSON con temperatura, setpoint, estado del calefactor, lectura lógica de sobrepresión y bandera de alarma. El handler de la API toma el mutex con un timeout de 80 ms, copia la estructura g_control_estado a una variable local y libera el mutex antes de formatear y enviar la respuesta; si el mutex no está disponible a tiempo, devuelve 503(servicio no disponible) con un cuerpo mínimo, evitando bloquear indefinidamente al cliente y evitando servir datos inconsistentes. Al terminar el arranque, la tarea se autoelimina con vTaskDelete(NULL), el portal sigue atendido por las tareas internas del stack HTTP de Espressif, mientras que web_ui_task ya no permanece en un bucle periódico como las tareas de ADC, control o display.

TAREA	PRIORIDAD	PERIODO O CONDICIÓN	STACK
task_control	5	Cada 100 ms	2048B
task_adc	4	Cada 200 ms	2048B
task_uart	3	Bloqueada en uart_read_bytes hasta recibir un carácter (portMAX_DELAY)	2048B
task_display	2	Cada 500 ms	2048B

web_ui_task	3	Un solo arranque: WiFi AP + httpd_start, luego vTaskDelete (no periódica)".	8192B
-------------	---	---	-------

4.4 Mecanismos de sincronización

Estructura de estado compartido

- La variable global es **g_control_estado**, de tipo **control_estado_t**.

typedef struct {

float temp_c; /* Temperatura medida en °C*/

int setpoint; /*Consigna en °C*/

bool alarm_active; /*Estado alarma*/

TickType_t alarm_low_since_ticks; /*Marca de ticks con I0.0 activada para salir de modo Alarma*/

bool calefactor_on;

bool sensor_sobrepresion;

} control_estado_t;

- **task_adc** escribe sólo temp_c.
- **task_control** lee temp_c y el estado de alarma. Actualiza alarm_active, alarm_low_since_ticks, setpoint, calefactor_on y sensor_sobrepresion.
- **task_display** sólo lee temp_c, setpoint y alarm_active para el OLED.
- El handler send_api en **web_ui.c** copia bajo mutex toda la estructura a una variable local st de tipo control_estado_t para armar el JSON.

Todo acceso a **g_control_estado** debe realizarse con el mutex tomado.

Mutex (g_control_mutex)

El mutex actúa como candado, sólo una tarea puede poseerlo a la vez.

Uso: xSemaphoreTake(g_control_mutex, timeout) → acceso a g_control_estado → xSemaphoreGive(g_control_mutex).

Timeouts: 50 ms (task_adc, task_display), portMAX_DELAY (task_control) y 80 ms en send_api(si no obtiene el mutex, responde 503).

Cola (queue_setpoint)

Es un buffer FIFO de FreeRTOS que transporta setpoints como enteros int en el rango 0–99 (tras la validación en task_uart) desde task_uart hasta task_control, sin que la UART escriba directamente en g_control_estado.

Se declara como QueueHandle_t estático en ESP32_FreeRTOS.c y se crea con

xQueueCreate(SETPOINT_QUEUE_LEN, sizeof(int)), donde SETPOINT_QUEUE_LEN vale 4: hasta cuatro mensajes pendientes.

task_uart: envía con xQueueSend(queue_setpoint, &setpoint, pdMS_TO_TICKS(50)). Si la cola está llena, espera como máximo 50 ms; si en ese plazo no pudo encolar, ese valor no se guarda para un reintento automático.

task_control: al inicio de cada ciclo hace xQueueReceive(queue_setpoint, &nuevo_setpoint, 0), el 0 indica sin espera, es decir no bloqueante. Si no hay mensaje nuevo, sigue con el setpoint local que ya tenía.

Las operaciones xQueueSend / xQueueReceive sobre la misma cola están pensadas para uso concurrente entre tareas en FreeRTOS: el kernel serializa el acceso a la cola, de modo que no hace falta un mutex adicional solo para encolar o desencolar el setpoint entre task_uart y task_control.

4.5 Manejo de errores

SITUACIÓN	DÓNDE SE DETECTA	ACCIÓN	EFEECTO
Fallo al crear mutex (g_control_mutex) o cola (queue_setpoint)	app_main() tras xSemaphoreCreateMutex() / xQueueCreate()	return (sin log en ese if)	No se ejecutan xTaskCreate ni web_ui_start(): el lazo de tareas no arranca.
Fallo uart_driver_install en UART0 (USB, setpoint)	app_main()	No se llama xTaskCreate(task_uart, ...); el resto de tareas sí se crean	No hay entrada de setpoint por terminal; task_control mantiene SETPOINT_DEFAULT (25 °C) hasta otra vía.
Fallo al crear unidad ADC1	board_esp32_init() (adc_oneshot_new_unit ≠ ESP_OK)	adc1_handle = NULL	board_esp32_adc_a0_0_mv() devuelve -1; leer_temperatura_lm35() puede quedar con n = 0 y devuelve 0.0 °C; el lazo puede reaccionar mal si se interpreta como “muy frío”.
Fallo calibración ADC	board_esp32_init() (adc_cali_create_scheme_line_fitting ≠ ESP_OK)	adc1_cali_a0 = NULL	Sigue habiendo lectura por conversión aproximada raw→mV; peor precisión, no bloquea el arranque.
Cola queue_setpoint llena al enviar setpoint	task_uart en xQueueSend(..., pdMS_TO_TICKS(50))	El resultado se ignora con (void)	Ese setpoint se pierde; el operador debe reenviar el comando.
Timeout al tomar g_control_mutex (50 ms)	task_adc	No escribe g_control_estado.temp_c en ese ciclo	La temperatura mostrada/controlada puede quedar desactualizada hasta el próximo ciclo exitoso.

Timeout al tomar g_control_mutex (50 ms)	task_display	vTaskDelay(200 ms) y continue	No se actualiza el OLED en esa pasada; reintento después del delay.
g_control_mutex nulo o timeout (80 ms) al tomar mutex	send_api (web_ui.c)	Responde 503	El navegador no recibe JSON de estado en esa petición (el script puede dejar valores viejos).
Fallo I2C en init del OLED	oled_display_init()	return antes de marcar éxito; oled_initialized permanece false (sin ESP_LOGE en esas líneas)	Display inoperativo hasta que otra llamada (p. ej. oled_show) reintente oled_display_init().
Fallo httpd_start	http_start() en web_ui.c	ESP_LOGE y return sin registrar URI	WiFi AP puede estar activo pero sin página / ni API /api/status útiles.
Fallos en init WiFi/NVS con ESP_ERROR_CHECK	wifi_init_ap() en web_ui.c	En IDF suele abortar el programa con mensaje de error	Reinicio o detención según configuración del proyecto; no es un fallo “silencioso” como el return de app_main.

4.6 Consideraciones de concurrencia

En este firmware varias actividades compiten por los mismos datos y recursos: las tareas task_adc, task_control y task_display, junto con los handlers HTTP del servidor en web_ui, pueden ejecutarse de forma entrelazada según el scheduler de FreeRTOS. Para evitar esto, todas las lecturas y escrituras de g_control_estado pasan por el mismo mutex g_control_mutex, con tiempos de espera acotados donde la interfaz puede ceder (50 ms en ADC y display, 80 ms en la API) y portMAX_DELAY en task_control, priorizando que el lazo y la lógica de alarma sigan actualizándose de forma predecible. El setpoint que llega

por UART no se escribe directamente en el estado compartido: task_uart solo encola enteros en queue_setpoint, y task_control los consume sin bloquear (xQueueReceive con espera cero), de modo que la recepción y validación del setpoint por UART0 no retiene el mutex ni prolonga la sección crítica del control. Las salidas físicas (GPIO de relés, cooler y buzzer) se actualizan después de liberar el mutex, reduciendo el tiempo en que otras tareas quedan bloqueadas esperando el mutex mientras se completan escrituras lentas a hardware. Las operaciones sobre la cola usan la API thread-safe del kernel, sin mutex adicional entre productor y consumidor del setpoint. Conviene tener presente que misma prioridad (por ejemplo task_uart y web_ui_task con prioridad 3) implica reparto por round-robin(se alternan), y que un handler HTTP que tardara demasiado podría retrasar a otras tareas de igual prioridad; el diseño actual limita el trabajo bajo mutex a copias breves de estructura y deja el formato JSON fuera de la sección crítica para mitigar ese riesgo.

5. Conclusiones

Este trabajo práctico atravesó las etapas típicas del desarrollo de un sistema embebido: primero con el montaje en PCB y validación de hardware se pudo asegurar el correcto funcionamiento de la placa, de modo que los problemas posteriores durante la implementación de software sean atribuidos a dicha etapa y no a fallas de cableado o periféricos.

A partir de este punto, la segunda parte, comenzando por el modelado con UML, ayudó a ordenar actores, responsabilidades, flujos de datos y estados, antes de comenzar con la programación propiamente dicha. Eso redujo ambigüedades sobre quien lee o escribe, temperatura, setpoint, alarma y salidas, y cómo se relacionan el operador, la UART, el display, la web y el lazo de control. En la implementación, esas decisiones se tradujeron en un diseño con varias tareas y prioridades, mutex sobre un estado compartido, y cola para el setpoint, además de temporizaciones y políticas de error acotadas. Se consolidó así la idea de concurrencia segura, donde el scheduler puede intercalar tareas, pero el acceso ordenado a datos y el paso de mensajes evitan condiciones de carrera y acoplamientos frágiles entre módulos.

Otro aprendizaje fue entender el uso del freeRTOS frente a bare-metal: en éste suele predominar un bucle principal e interrupciones, lo que complica atender tareas que deben ejecutarse con frecuencias distintas (muestreo, control, interfaz, red) sin que una parte bloquee otras y empuja a un estado global difícil de mantener. Con freeRTOS, la aplicación se organiza en tareas concurrentes con su período o bloqueo en E/S, el handler reparte el CPU según prioridad, y primitivas como mutex y colas estructuran al paralelismo lógico. El costo es mayor complejidad conceptual (sincronización, stacks, depuración de interacciones), pero el beneficio es un firmware más modular, escalable y alineado con sistemas industriales reales donde conviven control, comunicación e interfaz.

Por último, el uso de herramientas como Cursor resultó práctico para acelerar tiempos de codificación y corregir errores de sintaxis, pero no sustituye de ninguna manera el criterio del alumno, hubo que supervisar y rectificar constantemente nombres, tiempos, salidas incorrectas, asignaciones de prioridades, entre otras cosas. En un trabajo de ingeniería, la IA es una herramienta auxiliar, pero la responsabilidad del modelo, de la implementación y de la coherencia entre ambos sigue siendo del equipo que desarrolla y valida el sistema.

6. Referencias y código fuente

1. Espressif Systems. *ESP-IDF Programming Guide*. Documentación oficial ESP-IDF (build, FreeRTOS, WiFi, HTTP server). <https://docs.espressif.com/projects/esp-modbus/en/latest/esp32/>
2. FreeRTOS. *Mastering the FreeRTOS Real Time Kernel* — referencia de planificación, mutex, colas y buenas prácticas. https://www.freertos.org/Documentation/02-Kernel/07-Books-and-manual/01-RTOS_book.
3. Repositorio Proyecto_Final (archivos ESP32_FreeRTOS.c, web_ui.c, board_esp32.c, oled_display.c, control_state.c). https://github.com/nicojramos/Trabajo_Integrador_SEAI
4. UML-Diagrams.org, “Uml diagrams reference.” <https://www.uml-diagrams.org/>.